

DDA5001 Machine Learning

Regularization & Concluding Overfitting

Zhongxiang DAI

School of Data Science
The Chinese University of Hong Kong, Shenzhen



Recap: Cross Validation

- ▶ We need k to be small, so set $k = 1$.

$$\mathcal{S}_{\text{train}}^j = \{(\mathbf{x}_1, y_1), \dots, \cancel{(\mathbf{x}_j, y_j)}, \dots, (\mathbf{x}_n, y_n)\}$$

- ▶ Learn f'_j using $\mathcal{S}_{\text{train}}^j$.
- ▶ Validation error: $\text{Er}_{\text{val}}(f'_j) = e(f'_j(\mathbf{x}_j), y_j) := e_j$.
- ▶ Cross validation error:

$$\text{Er}_{\text{cv}} = \frac{1}{n} \sum_{j=1}^n e_j.$$

However, it has too many training rounds. This motivates us to consider **k -folds cross validation**, which chooses a batch of data as validation set at one time rather than exactly one data point.

Agenda

Regularization — Continued

Overfitting — Concluding Remarks

Regularization

- Validation is to estimate Er_{out} , and then adjust hyper-parameters.

The regularization is another weapon for eliminating overfitting, which penalizes the model complexity using penalty $\Omega(\mathcal{H})$:

$$\text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \Omega(\mathcal{H}), \quad \forall f \in \mathcal{H}$$

- From VC analysis, it is better to fit the data using the simplest workable $\mathcal{H}$. However, it is hard to determine such a perfect $\mathcal{H}$.
- One further heuristic extrapolation: How about use a rich/complex enough $\mathcal{H}$, but choose a 'simple' (the simplest workable) f from $\mathcal{H}$. Thus, we can choose a penalty $\Omega(f)$ to penalize the complexity of an individual f .
- Instead of minimizing $\text{Er}_{\text{in}}(f)$ alone, regularization amounts to minimizing simultaneously the training error and the complexity penalty, i.e.,

$$\min_{f \in \mathcal{H}} \text{Er}_{\text{in}}(f) + \Omega(f).$$

Least Square Revisited

Learning problem for least squares (linear regression)

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2,$$

where $\mathbf{X} \in \mathbb{R}^{n \times d}$.

When the data matrix $\mathbf{X}$ has full column rank (i.e., when its rank is d), we have a unique closed-form solution for LS:

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

- ▶ How about $n < d$?
- ▶ What is such a case? **Overfitting** occurs as d begins to exceed the number of samples n . We will get zero training error, but large test error.

Regularization Technique I: ℓ_2 -regularization

One candidate **regularizer**:

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2.$$

The ℓ_2 -regularized LS is:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

- ▶ The most direct way to reduce complexity is to let $\boldsymbol{\theta}$ have many zeros (recall its ‘VC dimension’ is d). But this is too hard a constraint. ℓ_2 -regularizer is to make some parameters small (close to zero).
- ▶ $\lambda > 0$ is the **regularization parameter** (a hyper-parameter) that controls the trade-off between underfitting and overfitting.
 - Too large λ results in underfitting.
 - Too small λ may lead to overfitting.
- ▶ **Validation** technique can be used to choose this hyper-parameter.
- ▶ We can apply ℓ_2 -regularization to logistic regression too.

Solution of ℓ_2 -regularization

Let

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2$$

Expanding the ℓ_2 -norms yields

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \lambda\boldsymbol{\theta}^\top \boldsymbol{\theta} \\ &= \mathbf{y}^\top \mathbf{y} + \boldsymbol{\theta}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\theta} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y}\end{aligned}$$

Taking the gradient

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = 2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\theta} - 2\mathbf{X}^\top \mathbf{y}$$

Setting the gradient to zero gives

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

ℓ_2 -regularization vs. Vanilla Least Squares

Least squares:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 \\ &= \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y} - \text{only for full column rank case}\end{aligned}$$

ℓ_2 -regularization:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2 \\ &= \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^\top \mathbf{y}\end{aligned}$$

The advantage of ℓ_2 -regularization:

$$\underbrace{\left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1}}_{\text{always invertible}}$$

Algebraically explained why ℓ_2 -regularization is useful.

Weight Decay

Weight decay is an important technique in machine learning. It is used almost everywhere in the training of neural networks.

- ▶ Weight decay is proposed as a technique for directly decaying the parameter (weight) θ during the algorithm process. It has the form:

$$\theta_{k+1} = (1 - \lambda)\theta_k - \mu \nabla \mathcal{L}(\theta_k),$$

where λ defines the rate of the weight decay per step.

- ▶ It is easy to see that if $1 - \lambda \in (0, 1)$, the weight parameter θ is decaying at each iteration, thus the name weight decay.

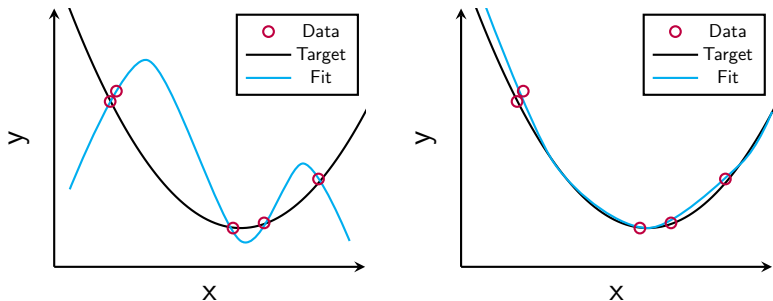
Indeed, we can verify that weight decay is equivalent to **applying gradient descent to the ℓ_2 -regularized problem**:

$$\min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta) + \frac{\lambda'}{2} \|\theta\|_2^2, \quad \text{with} \quad \lambda' = \frac{\lambda}{\mu}.$$

However, this is **NOT** the case in the **Adam** algorithm (later); see [1].

[1] Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. ICLR 2019.

Regularization as a Cure for Overfitting



- ▶ Left: Using fourth-order polynomial **without regularization**.
- ▶ Right: Using fourth-order polynomial with **regularization (weight decay)**.

Regularization Technique II: ℓ_1 -regularization / Lasso

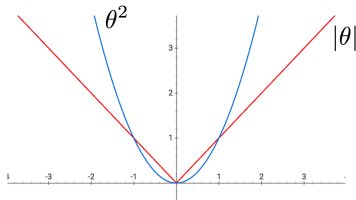
An alternative regularizer to ℓ_2 -regularization:

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^d |\theta_i|,$$

which is called *least absolute shrinkage and selection operator* (**Lasso**) or simply **ℓ_1 -regularization**.

Properties of ℓ_1 -norm:

- **Fact:** Promotes sparsity.
- Not differentiable.



The Lasso Problem

The Lasso problem/ ℓ_1 -regularized LS:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \quad \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1$$

Compared to ℓ_2 -regularization:

- ▶ Lasso promotes sparsity in a more explicit way.
 - It explains why Lasso can prevent overfitting when $n < d$ as we indeed have **much less parameters than d (due to sparsity, many zeros in $\boldsymbol{\theta}$)**.
- ▶ We can also apply ℓ_1 -regularization to logistic regression.

The issue is how to solve the Lasso problem.

Can we apply GD to Lasso? What will be the problem?

Proximal Gradient Descent

Algorithm Design Framework Revisited ?

Suppose the task is $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$, we can design an algorithm as

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ l_k(\boldsymbol{\theta}) = q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

μ_k is stepsize-like quantity.

- ▶ When $q_k(\boldsymbol{\theta})$ is linear approximation of $\mathcal{L} \implies$ **gradient descent**
- ▶ When $q_k(\boldsymbol{\theta})$ is $\mathcal{L}$ itself $\implies$ **proximal point method**

How to design an iterative algorithm for solving the Lasso problem?

The idea:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = g(\boldsymbol{\theta}) + \Omega(\boldsymbol{\theta}) = \underbrace{\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2}_{\text{linear approx.}} + \underbrace{\lambda \|\boldsymbol{\theta}\|_1}_{\text{keep itself}}$$

Proximal Gradient Descent for The Lasso

Proximal gradient descent for Lasso

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ l_k(\boldsymbol{\theta}) = \underbrace{g(\boldsymbol{\theta}_k) + \nabla g(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k)}_{q_k} + \lambda \|\boldsymbol{\theta}\|_1 + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

The principle behind:

- ▶ Use linear approximation for differentiable part (gradient descent).
- ▶ Use the function itself for nondifferentiable part (proximal point method).

Overall,

proximal point + gradient descent $\implies$ proximal gradient descent (PGD)

The Update

For the ℓ_1 -regularizer, PGD has a closed-form update, which is given by

$$\theta_{k+1}[i] = \begin{cases} \alpha[i] - \lambda\mu_k, & \text{if } \alpha[i] \geq \lambda\mu_k \\ 0, & \text{if } -\lambda\mu_k < \alpha[i] < \lambda\mu_k \\ \alpha[i] + \lambda\mu_k, & \text{if } \alpha[i] \leq -\lambda\mu_k \end{cases}$$

for $i = 1, \dots, d$, where

$$\alpha = \theta_k - \mu_k \nabla g(\theta_k)$$

- ▶ This closed-form update is derived in the [appendix](#) of this lecture.
- ▶ We also give a quick general PGD extension in the appendix.

The Trade-off in Regularization

Underfitting and Overfitting

Regularization hopes to release the use of **complex model** and then penalize it to the right / lower complexity, leading to correct fit of data.

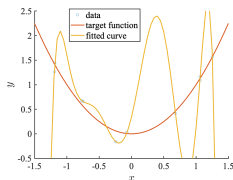
- ▶ In practice, choosing a Ω is often a **heuristic**.
- ▶ Finding a perfect Ω can be as difficult as finding a perfect $\mathcal{H}$ as it depends on the information that, by the very nature of learning, we do not have, namely d_{VC}^* .
- ▶ Fortunately, some long-standing regularizers work well across many applications such as ℓ_2 - and ℓ_1 -regularizers.
- ▶ ℓ_2 -regularizer (weight decay) is the most widely used one in training neural networks.

Even in this case, the amount of regularization (controlled by λ) leverages/controls overfitting and underfitting. Too much regularization might lead to **underfitting** and vice versa.

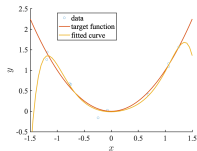
Example: Effect of Different Level of Regularization

Fit a few noisy data generated by **quadratic** target model, using **10-th order polynomial** with **weight decay**. We have all three catalysts causing overfitting.

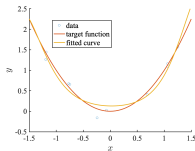
$$\lambda = 0$$



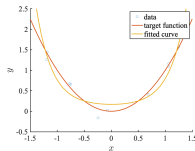
$$\lambda = 0.1$$



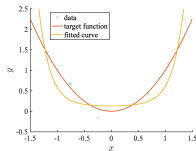
$$\lambda = 0.8$$



$$\lambda = 2$$



$$\lambda = 10$$



Regularization is to Mitigate Noise

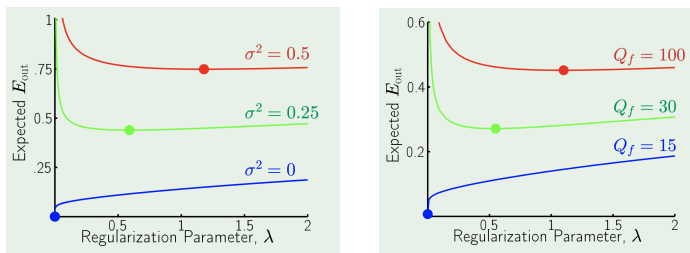


Figure: Left: σ^2 represents noise level; Right: Q_f is the target model complexity of the underlying g . Here, the used model complexity to fit is 15.

- ▶ The added noise in the left can be regarded as **stochastic noise**, while the complexity mismatch in the right can be regarded as **deterministic noise**. Both noises have similar effect on overfitting.
- ▶ In the **noiseless** case, we need **no regularization**. With the increase of noise, we have worse performance (model starts to fit the noise), and the optimal regularization parameter also increases as we need more regularization to prevent the fitting of more noise.
- ▶ Regularization helps by reducing the impact of the noise.

Regularization and VC Dimension

VC line of reasoning for regularization:

- ▶ As λ increases, the learning algorithm changes. However, the d_{VC} keeps unchanged (since $\mathcal{H}$ remains unchanged).
- ▶ Indeed, even though $\mathcal{H}$ is not changed, more regularization leads to an 'effectively small' model, which generalize better.
- ▶ A heuristic in practice is to use 'effective VC dimension':
 - ▶ In linear classifier, the VC dimension is $d + 1$, equals to the number of parameters.
 - ▶ Similarly, the 'effective VC dimension' is the effective number of parameters, i.e., the number of nonzero parameters. That is, with regularization, we're effectively searching in a subset of $\mathcal{H}$ with smaller complexity, characterized by a smaller 'effective VC dimension'.
- ▶ This gives a good reasoning that regularization gives better generalization ability: Though $\mathcal{H}$ is not changed, the algorithm tends to find a simpler f with a smaller complexity:

$$\text{Er}_{\text{out}}(f_{\hat{\theta}}) \leq \text{Er}_{\text{in}}(f_{\hat{\theta}}) + \underbrace{\text{generalization of } f_{\hat{\theta}}}_{\text{refined by regularization}}$$

Agenda

Regularization — Continued

Overfitting — Concluding Remarks

▪

Theory and Practice

Theory versus Practice

- ▶ Validation and regularization present **challenges** for the theory of generalization analysis. It is not straightforward to conduct a **rigorous** VC analysis for validation, cross validation, and regularization.
- ▶ What is indeed quite effective is to **use theory to guide practice** (what we have done): Regularization constrains the effective model complexity of the **individual learned f** and hence leads to better generalization (though $\mathcal{H}$ does not change), while validation **roughly** estimates and bounds $\text{Er}_{\text{out}}(\hat{f})$.
- ▶ Learning from data is an **empirical** task with theoretical underpinnings. The only way to be convinced what works and what does not is to implement them on real applications.

▪

Data Snooping

Data Snooping

Data Snooping

If a dataset has affected any step in the learning process, its ability to assess the outcome has been compromised.

- ▶ This is by far the most common trap that people fall into in practice.
- ▶ It is extremely important to choose the learned model **before** seeing any test data.
- ▶ Otherwise, this can lead to serious overfitting.
- ▶ It can be very subtle. People may be trapped without awareness.
- ▶ Many ways to slip up: Reuse of test data set, etc.

Example: Data Snooping

Pretraining on the Test Set Is All You Need

Rylan Schaeffer

September 19, 2023

Abstract

Inspired by recent work demonstrating the promise of smaller Transformer-based language models pretrained on carefully curated data, we supercharge such approaches by investing heavily in curating a novel, high quality, non-synthetic data mixture based solely on evaluation benchmarks. Using our novel dataset mixture consisting of less than 100 thousand tokens, we pretrain a 1 million parameter transformer-based LLM **phi-CTNL** (pronounced "fictional") that achieves perfect results across diverse academic benchmarks, strictly outperforming all known foundation models. **phi-CTNL** also beats power-law scaling and exhibits a never-before-seen grokking-like ability to accurately predict downstream evaluation benchmarks' canaries.

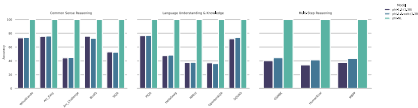


Figure 1: Benchmark results comparing **phi-CTNL** and other state-of-the-art open-source LLMs. Benchmarks are broadly classified into three categories: common sense reasoning, language skills, and multi-step reasoning. The classification is meant to be taken loosely. One can see that **phi-CTNL** achieves perfect scores, smashing current state-of-the-art on all benchmarks. Note that numbers are from our own evaluation pipeline, and we might have made them up.

4 Discussion

We introduced **phi-CTNL**, a 1 million parameter LLM, trained primarily on a specially curated non-synthetic dataset of 100 thousand tokens. Our findings suggest that **phi-CTNL** far surpasses all known models on academic evaluations while using several orders of magnitude fewer parameters and pretraining tokens. This result challenges the prevailing notion that the capabilities of LLMs at solving academic benchmarks are solely determined by their parameter scale, suggesting that data quality plays an even more important role than previously thought.

Disclaimer: if you haven't figured out by now that this manuscript is satire, this manuscript is satire. Please see this Twitter thread for more information and discussion. It is this author's belief that while language model evaluation and benchmarking is hard work, and oftentimes unglamorous, the field is generally undermined by boastful claims made without serious investigation of data contamination risks. This author does appreciate work like **phi-1** [GZA'23], **TinyStories** [EL23] and **phi-1.5** [LBE'23] that studies how to construct pretraining corpora aimed at sample-efficient learning.

arXiv:2309.08632v1 [cs.CL] 13 Sep 2023

Summary of Overfitting

- ▶ Noise, complexity of $\mathcal{H}$, number of data points affect learning, and may lead to overfitting.
- ▶ Validation is a technique for estimating $E_{r_{out}}$, giving a way for choose hyper-parameter to avoid overfitting.
- ▶ Regularization is to penalize the individual learned model complexity, reducing overfitting issue (caused by stochastic and deterministic noises).

Appendix

Closed-form Update of Proximal Gradient Descent for Lasso

The Update

Proximal gradient descent

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ l_k(\boldsymbol{\theta}) = \underbrace{g(\boldsymbol{\theta}_k) + \nabla g(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k)}_{q_k} + \lambda \|\boldsymbol{\theta}\|_1 + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

How to solve this subproblem?

Combing the quadratic term and the linear term, we can rewrite the proximal gradient descent as

$$(\text{PGD}) \quad \boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{2\mu_k} \|\boldsymbol{\theta} - (\boldsymbol{\theta}_k - \mu_k \nabla g(\boldsymbol{\theta}_k))\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1.$$

The Update

PGD:

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{2\mu_k} \|\boldsymbol{\theta} - (\boldsymbol{\theta}_k - \mu_k \nabla g(\boldsymbol{\theta}_k))\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1$$

The key feature of the subproblem: **decomposable**.

Let

$$\boldsymbol{\alpha} = \boldsymbol{\theta}_k - \mu_k \nabla g(\boldsymbol{\theta}_k)$$

Denote $\theta[i]$ as the i -th coordinate of $\boldsymbol{\theta}$. PGD can be written as

$$\begin{aligned} \boldsymbol{\theta}_{k+1} &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\alpha}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \\ &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \sum_{i=1}^d \left[\frac{1}{2\mu_k} (\theta[i] - \alpha[i])^2 + \lambda |\theta[i]| \right] \\ &= \sum_{i=1}^d \operatorname{argmin}_{\theta[i] \in \mathbb{R}} \frac{1}{2\mu_k} (\theta[i] - \alpha[i])^2 + \lambda |\theta[i]|, \end{aligned}$$

which is reduced to $d \times$ one-dimensional optimization problems.

The Update

Finally, we have (small exercise)

$$\begin{aligned}\theta_{k+1}[i] &= \operatorname{argmin}_{\theta[i] \in \mathbb{R}} \frac{1}{2\mu_k} (\theta[i] - \alpha[i])^2 + \lambda |\theta[i]| \\ &= \begin{cases} \alpha[i] - \lambda\mu_k, & \text{if } \alpha[i] \geq \lambda\mu_k \\ 0, & \text{if } -\lambda\mu_k < \alpha[i] < \lambda\mu_k \\ \alpha[i] + \lambda\mu_k, & \text{if } \alpha[i] \leq -\lambda\mu_k \end{cases}\end{aligned}$$

We have closed-form update for PGD used to solve Lasso.

Extension: PGD for General Regularized Problems

Consider general regularized problem

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = g(\boldsymbol{\theta}) + \Omega(\boldsymbol{\theta}).$$

Define the **proximal mapping** of function Ω as

$$\text{prox}_{\alpha\Omega}(\boldsymbol{\alpha}) = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\text{argmin}} \frac{1}{2\alpha} \|\boldsymbol{\theta} - \boldsymbol{\alpha}\|_2^2 + \Omega(\boldsymbol{\theta}).$$

► For Lasso, $\Omega(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$ and

$$\boldsymbol{\theta}_{k+1} = \text{prox}_{\mu_k\Omega}(\boldsymbol{\theta}_k - \mu_k \nabla g(\boldsymbol{\theta}_k)).$$

PGD for general regularized problem

$$\boldsymbol{\theta}_{k+1} = \text{prox}_{\mu_k\Omega}(\boldsymbol{\theta}_k - \mu_k \nabla g(\boldsymbol{\theta}_k)).$$

The design principle: The above proximal mapping update has closed-form solution.